

Generated for epistemic-ontology.net from `README.md` in the Record Harm Ontology repository. Term IRIs are the permanent ones under `https://www.epistemic-ontology.net/record-harm#`; the repository itself still drafts under a placeholder base, so queries copied from the repository will differ.

Record Harm Ontology

A formal OWL 2 DL ontology for modeling ontological attacks on informational records.

version **3.1**

OWL **2 DL**

(<https://www.w3.org/TR/owl2-overview/>)

SHACL **validated**

OWL 2 DL note: `ex:buildsUpon` is asymmetric and irreflexive and is used in `ex:CompositeHarm`'s cardinality-based definition, so it is deliberately **not** declared transitive — a transitive property is "non-simple" in OWL 2 DL and may not be asymmetric/irreflexive or appear in cardinality restrictions. Transitive dependency chains are computed on demand with the SPARQL property path `ex:buildsUpon+` (see [docs/QUERIES.md](#)), not materialized by the reasoner. This keeps the ontology inside OWL 2 DL so a conforming reasoner can run the disjointness and CompositeHarm-membership cross-checks.

Overview

The Record Harm Ontology provides a rigorous taxonomy of how informational records can be damaged, destroyed, or corrupted. It distinguishes between **prime harms** (ontologically irreducible attacks) and **composite harms** (derived from combinations of primes), while also modeling specific harm events and recurring harm patterns.

This ontology serves multiple communities:

For Information Security Practitioners

If you work with **STRIDE**, **CIA Triad**, or **threat modeling**, this ontology provides:

- **Finer-grained integrity analysis** than CIA's broad "Integrity" category (6 distinct prime harms)
- **Evidence chain tracking** for forensics and incident response
- **Formal dependency modeling** showing how complex attacks build from primitives

- **Direct mappings** to STRIDE categories (see [ONTOLOGY-COMPARISON.md](#))

Your familiar concepts: Tampering → Fabrication/Alteration, DoS → Destruction/Suppression, Repudiation → Denial/Repudiation

For Social Scientists and Conflict Researchers

If you study **disinformation, propaganda, historical revisionism, or social conflict**, this ontology provides:

- **Systematic classification** of information manipulation tactics
- **Pattern recognition** for coordinated campaigns (cover-ups, miscontextualization)
- **Community-level record modeling** (collective memory, institutional archives)
- **Tracking harm to shared narratives** and communal knowledge

Your domain: Genocide denial (Denial), historical erasure (Destruction), propaganda (Fabrication/Contamination), archival suppression (Suppression), selective history (Omission)

For AI Researchers and Psychologists

If you work with **AI agents, cognitive modeling, or memory systems**, this ontology provides:

- **Agent-centric record modeling** (memories, beliefs, internal states)
- **Self-directed harm representation** (repression, self-deception, confabulation)
- **Inside-agent vs. inside-community distinction** (individual vs. collective knowledge)
- **Novel applications** to AI alignment, belief revision, and epistemic integrity

Your domain: Memory distortion (Alteration), motivated forgetting (Destruction), confabulation (Fabrication), repression (Suppression), belief denial (Denial)

Key Features

- **6 Prime Harms:** Destruction, Fabrication, Alteration, Omission, Denial, Suppression
- **6 Composite Harms:** Built from formal dependencies on prime harms
- **Event Layer:** Model specific occurrences with dates, perpetrators, and severity
- **Pattern Layer:** Capture empirical co-occurrences of independent harms

- **SHACL Validation:** Comprehensive constraint checking separate from OWL reasoning
- **SKOS Vocabularies:** Controlled terms for aspects, detectability, and reversibility
- **Record Location Modeling:** Inside-agent (memories, beliefs) and inside-community (registers, archives)

Quick Start by Domain

InfoSec: Map Your Threat Model

```
# Find all harms that map to STRIDE Tampering
PREFIX ex: <https://www.epistemic-ontology.net/record-harm#>
SELECT ?harm ?label WHERE {
    VALUES ?harm { ex:Fabrication ex:Alteration ex:Contamination }
    ?harm rdfs:label ?label .
}
```

Next: See [ONTOLOGY-COMPARISON.md](#) for complete STRIDE/CIA mappings

Social Science: Track Disinformation Campaigns

```
ex:GenocideDenialCampaign a ex:HarmPattern ;
    rdfs:label "Genocide Denial Campaign" ;
    ex:includesHarm ex:Denial, ex:Suppression, ex:Fabrication ;
    rdfs:comment "Coordinated denial + evidence suppression + counter-narrative fa
```

Next: See communal examples below for collective memory modeling

AI/Psychology: Model Agent Memory

```
ex:AgentMemory a ex:Record ;
    ex:bearer ex:AIAgent ;
    rdfs:label "Agent's belief about event X" .

ex:BeliefRevision a ex:HarmEvent ;
    ex:ofType ex:Alteration ;
```

```
ex:harms ex:AgentMemory ;  
ex:perpetrator ex:AIAgent ; # Self-directed  
ex:severity 3 .
```

Next: See agent-centric examples below for repression, confabulation, self-deception

Use Cases

- **Digital Forensics:** Evidence integrity analysis and chain of custody tracking
- **Records Management:** Archival science and preservation planning
- **Information Security:** Threat modeling and attack surface analysis
- **Legal/Regulatory:** Compliance with record-keeping requirements
- **Misinformation Analysis:** Tracking disinformation campaigns and propaganda
- **Historical Research:** Documenting record manipulation and censorship
- **Psychological Research:** Modeling memory distortion, repression, and self-deception
- **Community Governance:** Tracking testimony admission and register integrity
- **AI Alignment:** Epistemic integrity for AI agents and belief systems
- **Conflict Studies:** Systematic analysis of information warfare tactics

Examples: Agent-Centric and Communal Records

The v3.1 revision introduces explicit modeling of **where records live** — inside individual agents (memories, beliefs) or inside communities (registers, archives). This enables representation of harms that were previously inexpressible.

Agent-Centric (Psychological) Examples

Repression — A witness to a traumatic event actively suppresses the memory:

```
ex:TraumaMemory a ex:Record ;  
  rdfs:label "Memory of the incident" ;  
  ex:bearer ex:Witness .
```

```
ex:RepressionEvent a ex:HarmEvent ;
  ex:ofType ex:Suppression ;
  ex:harms ex:TraumaMemory ;
  ex:perpetrator ex:Witness ; # Self-directed harm
  ex:severity 7 .
```

The perpetrator and bearer are the same agent — this is **self-directed harm**, newly expressible in v3.1.

Confabulation — An agent fabricates a false memory to fill a gap:

```
ex:FalseMemory a ex:Record ;
  rdfs:label "Confabulated memory of approval" ;
  ex:bearer ex:Executive .

ex:ConfabulationEvent a ex:HarmEvent ;
  ex:ofType ex:Fabrication ;
  ex:harms ex:FalseMemory ;
  ex:perpetrator ex:Executive ; # Self-directed
  ex:severity 5 .
```

Self-Deception — An agent denies their own memory to avoid cognitive dissonance:

```
ex:InconvenientMemory a ex:Record ;
  rdfs:label "Memory contradicting self-image" ;
  ex:bearer ex:Agent .

ex:SelfDeceptionEvent a ex:HarmEvent ;
  ex:ofType ex:Denial ;
  ex:harms ex:InconvenientMemory ;
  ex:perpetrator ex:Agent ; # Self-directed
  ex:severity 4 .
```

Motivated Forgetting — Deliberately allowing a memory to decay:

```
ex:UnwantedMemory a ex:Record ;
  ex:bearer ex:Agent .
```

```
ex:MotivatedForgettingEvent a ex:HarmEvent ;
  ex:ofType ex:Destruction ;
  ex:harms ex:UnwantedMemory ;
  ex:perpetrator ex:Agent ; # Self-directed
  ex:severity 6 .
```

Communal Examples

Parish Register — A community register as a record composed of individual testimonies:

```
ex:ParishRegister a ex:Record ;
  rdfs:label "Parish baptism register" ;
  ex:bearer ex:ParishCommunity ;
  ex:hasElement ex:Testimony1, ex:Testimony2, ex:Testimony3 .

ex:Testimony1 a ex:Record ;
  rdfs:label "John's testimony of baptism" ;
  ex:bearer ex:John .
```

Refused Testimony — A community refuses to admit an agent's testimony into the register:

```
ex:RefusedTestimony a ex:Record ;
  rdfs:label "Mary's testimony (refused)" ;
  ex:bearer ex:Mary .

ex:RefusalEvent a ex:HarmEvent ;
  ex:ofType ex:Omission ;
  ex:harms ex:ParishRegister ; # Omission FROM the register
  ex:perpetrator ex:ParishClerk ;
  ex:severity 7 ;
  rdfs:comment "Mary's testimony was kept out of the register" .
```

Note: The harm is to the **register** (which is incomplete), not to Mary's testimony (which remains intact but excluded).

Collective Memory Suppression — A community collectively suppresses knowledge of an event:

```
ex:CommunityMemory a ex:Record ;
  rdfs:label "Collective memory of the incident" ;
```

```
ex:bearer ex:Community1, ex:Community2, ex:Community3 . # Multiple bearers

ex:CollectiveSuppressionEvent a ex:HarmEvent ;
  ex:ofType ex:Suppression ;
  ex:harms ex:CommunityMemory ;
  ex:perpetrator ex:CommunityLeadership ;
  ex:severity 8 .
```

Archival Fragmentation — A collection is deliberately scattered across institutions:

```
ex:ArchivalCollection a ex:Record ;
  rdfs:label "Complete correspondence archive" ;
  ex:hasElement ex:Letter1, ex:Letter2, ex:Letter3 .

ex:FragmentationEvent a ex:HarmEvent ;
  ex:ofType ex:Fragmentation ;
  ex:harms ex:ArchivalCollection ;
  ex:perpetrator ex:Archivist ;
  ex:severity 6 ;
  rdfs:comment "Letters dispersed to prevent reconstruction of the full narrative"
```

Key Insight: No New Harm Types Needed

The inside-agent / inside-community distinction required **no new harm types**. Every case maps to an existing prime:

SCENARIO	HARM TYPE	NOTES
Forgetting	Destruction	Memory permanently lost
Confabulation	Fabrication	False memory created
Self-deception	Denial	Agent denies own memory
Repression	Suppression	Memory intact but inaccessible
Refused testimony	Omission	Excluded from community register

Location is a property of the **record** (`ex: bearer`), not a dimension of harm.

Bridging Vocabularies: Your Domain → Record Harm

From InfoSec Terminology

YOU SAY	WE SAY	WHY IT MATTERS
Tampering	Fabrication + Alteration	Distinguishes creating false vs. modifying genuine
DoS	Destruction + Suppression	Permanent loss vs. temporary concealment
Integrity violation	6 prime harms	Finer granularity for forensic analysis
Chain of custody	ForgeryOfProvenance	Explicit composite harm with dependencies
Evidence spoliation	Destruction + Suppression + Alteration	Pattern-based modeling

What's missing: Confidentiality (deliberate—leaked records are undamaged). Add your own layer if needed.

From Social Science Terminology

YOU SAY	WE SAY	WHY IT MATTERS
Historical revisionism	Alteration + Omission + Fabrication	Tracks specific manipulation tactics
Propaganda	Fabrication + Contamination	Distinguishes pure fabrication from mixing genuine/false
Censorship	Suppression + Destruction	Separates hiding from erasing

YOU SAY	WE SAY	WHY IT MATTERS
Collective memory	Record with multiple bearers	Explicit community-level modeling
Narrative control	HarmPattern	Captures coordinated campaigns
Genocide denial	Denial + Suppression + Fabrication	Systematic pattern recognition

What's new: Inside-community records (registers, archives) as first-class entities with mereology.

From AI/Psychology Terminology

YOU SAY	WE SAY	WHY IT MATTERS
Memory distortion	Alteration (inside-agent)	Same harm type, different bearer
Confabulation	Fabrication (self-directed)	perpetrator = bearer
Repression	Suppression (self-directed)	Memory intact but inaccessible
Motivated forgetting	Destruction (self-directed)	Deliberate memory loss
Belief revision	Alteration of belief-record	Beliefs are records with agent bearers
Cognitive dissonance	Denial (self-directed)	Denying one's own memory

What's new: Self-directed harm (perpetrator = bearer) is explicitly representable. No other framework models this.

Comparison with Other Frameworks

See <docs/ONTOLOGY-COMPARISON.md> for detailed mappings to:

- **STRIDE** (Microsoft threat modeling)
- **CIA Triad** (Confidentiality, Integrity, Availability)

- **InterPARES Diplomatics** (archival authenticity)
- **Spoliation Doctrine** (legal evidence destruction)

Key differences: Record Harm is **record-centric** (not system-centric), includes **inside-agent records** (memories, beliefs), and models **formal dependencies** between harms.

Repository Structure

```
record-harm-ontology/  
├── ontology/  
│   └── record-harm-ontology.ttl    # Main OWL 2 DL ontology (v3.1)  
├── shapes/  
│   └── record-harm-shapes.ttl     # SHACL validation constraints  
├── examples/  
│   └── example-harm-events.ttl    # Worked HarmEvent / Record / Agent data  
├── docs/  
│   ├── ARCHITECTURE.md           # Design patterns and rationale  
│   └── QUERIES.md                # Example SPARQL queries  
├── scripts/  
│   └── validate.py               # Syntax + SHACL validation + metrics  
├── CHANGELOG.md  
├── CONTRIBUTING.md  
├── LICENSE  
└── README.md
```

Quick Start

Loading the Ontology

Python (rdflib):

```
from rdflib import Graph  
  
g = Graph()  
g.parse("ontology/record-harm-ontology.ttl", format="turtle")  
  
# Query for all prime harms
```

```

query = """
PREFIX ex: <https://www.epistemic-ontology.net/record-harm#>
SELECT ?harm ?label WHERE {
    ?harm a ex:PrimeHarm ;
        rdfs:label ?label .
}
"""

for row in g.query(query):
    print(f"{row.harm}: {row.label}")

```

SPARQL Endpoint:

```

PREFIX ex: <https://www.epistemic-ontology.net/record-harm#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>

# Find all harms targeting Authenticity
SELECT ?harm ?label ?definition WHERE {
    ?harm ex:targetsAspect ex:Authenticity ;
        rdfs:label ?label ;
        skos:definition ?definition .
}

```

Validating with SHACL

Python (pyshacl):

```

from pyshacl import validate

data_graph = "path/to/your/data.ttl"
shacl_graph = "shapes/record-harm-shapes.ttl"
ontology_graph = "ontology/record-harm-ontology.ttl"

conforms, results_graph, results_text = validate(
    data_graph,
    shacl_graph=shacl_graph,
    ont_graph=ontology_graph,
    inference='rdfs',
    abort_on_first=False
)

```

```
print(f"Conforms: {conforms}")
print(results_text)
```

Ontology Architecture

Type Layer (RecordHarm Taxonomy)

The ontology defines 12 harm types organized into two disjoint classes:

Prime Harms (ontologically irreducible):

1. **Destruction** - Complete annihilation of the record
2. **Fabrication** - Creation of false records
3. **Alteration** - Modification of existing genuine records
4. **Omission** - Deliberate exclusion of relevant elements, *including* relations, ordering, and adjacent metadata
5. **Denial** - Refusing to acknowledge existence/validity
6. **Suppression** - Severing the record ↔ observer channel while the record stays intact

Composite Harms (built from primes via `ex:buildsUpon`):

1. **ForgeryOfProvenance** - Falsifying origin/custody (builds on Fabrication + Alteration)
2. **Fragmentation** - Breaking records into disconnected pieces (builds on Omission)
3. **Obfuscation** - Degrading a record's form so it resists understanding (builds on Suppression + Alteration)
4. **Decontextualization** - Stripping metadata/context (builds on Omission)
5. **Contamination** - Mixing genuine with fabricated (builds on Fabrication)
6. **Repudiation** - Disavowing authorship/authority (builds on Denial + Fabrication)

An `ex:buildsUpon` edge asserts a **necessary presupposition**, not a frequent co-occurrence.

Harms that merely travel together belong in `ex:HarmPattern` (`CoverUpPattern` , `MiscontextualizationPattern`).

Why six primes and not five

A candidate prime has to pass two tests: no other harm in the taxonomy reduces it, **and** it is the only route to some aspect of the record it attacks. Suppression passes both. It does not reduce to Omission — a suppressed record is complete and unmodified, so nothing has been excluded from it; only the channel to an observer is cut. It does not reduce to Denial either — a classified file is openly acknowledged to exist, so the custodian denies nothing and withholds everything. And it is the only prime targeting `ex:Accessibility`; before v3.0 no prime targeted that aspect at all, which is why Suppression had been hung off Omission with an edge that contradicted both definitions. `scripts/validate.py` now enforces the coverage property so this class of defect cannot recur silently.

Where a Record Lives (v3.1)

`ex:Record` was a terminal node until v3.1 — the object of `ex:harms` and the subject of nothing — so nothing could say what a record is made of or who holds it, while three harm definitions were already quantifying over exactly that in prose (`Omission` over a record's "elements", `Fragmentation` over its "pieces", `Suppression` over the channel to "an observer").

Two properties close that:

- `ex:hasElement` / `ex:elementOf` — mereology, aligned to `dcterms:hasPart` / `isPartOf`. A community's register is an `ex:Record` in its own right whose elements are the records individual agents brought to it. Asymmetric and irreflexive, **not** transitive (the v2.3 precedent); walk nesting with `ex:hasElement+`.
- `ex:bearer` / `ex:bears` — where the record resides. One individual bearer is *inside-agent* (a memory, a belief, a private ledger); many bearers, or one collective bearer, is *inside-community*.

The inside-agent / inside-community distinction needed no new harm type and no new aspect. Every case lands on an existing prime:

CASE	HARM
forgetting	Destruction
confabulated memory	Fabrication
self-deception, disavowing one's own memory	Denial

CASE	HARM
repression — knowing and refusing to surface it	Suppression
testimony kept out of a community's register	Omission <i>from that register</i>

So location is a parameter of the **record**, not a dimension of harm. The six primes are unchanged.

What is newly expressible is **self-directed harm**: a `HarmEvent` whose `ex:perpetrator` is also the `ex:bearer` of the record it harms. Repression and motivated forgetting had no representation before v3.1 (query in [docs/QUERIES.md](#)).

Two things documented rather than fixed

- **detectability / reversibility are calibrated to community-borne records.** `Denial` is tagged `EasilyDetectable` — true of a record in a shared register you can point at, false of an agent denying its own memory, which no one can audit. `Destruction` is tagged `Irreversible` — true inside an agent, overstated for a redundantly held community record. A proper fix needs bearer-relative qualified values; `severity` dodged the same problem in v2.2 by living on the event rather than the type.
- **Pure non-creation stays out of scope.** An agent who witnesses an event and records nothing harms no record, because there is no record. The community-side case *is* covered (non-admission is Omission from the register). Deciding the first would require a notion of the record that *ought* to exist, which nothing here supplies.

Deliberate non-coverage: confidentiality

There is no Confidentiality aspect and no disclosure harm, so this vocabulary does **not** map one-to-one onto STRIDE or the CIA triad — Information Disclosure has no counterpart here. A leaked record is undamaged *as a record*: it exists, and is authentic, complete, accessible, contextualized and trustworthy; it may be more probative after the leak than before. The injury runs to the record's subject or custodian, which is a different ontology than this one.

Event Layer (HarmEvent)

Model specific occurrences:

```
ex:exampleDestructionEvent a ex:HarmEvent ;
    ex:ofType ex:Destruction ;
```

```
ex:harms ex:SomeSpecificRecord ;  
dc:date "2026-06-15"^^xsd:date ;  
ex:perpetrator ex:SomeAgent ;  
ex:severity 8 .
```

Pattern Layer (HarmPattern)

Capture recurring combinations:

```
ex:CoverUpPattern a ex:HarmPattern ;  
rdfs:label "Cover-up" ;  
ex:includesHarm ex:Suppression, ex:Alteration, ex:Denial .
```

Record Aspects

Every harm targets one or more of six fundamental aspects:

- **Existence** - The record's being or presence
- **Authenticity** - Genuine link to claimed origin/content
- **Integrity** - Wholeness and completeness
- **Accessibility** - Discoverability and comprehension
- **Context** - Provenance and surrounding circumstances
- **Trustworthiness** - Reliability and legitimacy

Validation Rules (SHACL)

The ontology includes comprehensive SHACL shapes that enforce:

- Prime harms must have zero `buildsUpon` edges
- Composite harms must have at least one `buildsUpon` edge
- Every harm must target at least one aspect
- Every harm must be classified as exactly one of Prime/Composite
- Harm events must reference exactly one harm type

- Severity values must be integers 1-10
- Detectability/reversibility must use controlled vocabulary terms

Plus `ex:RecordShape` (elements of a record are records; bearer uncapped) and `ex:RecordAcyclicShape`, a SPARQL-based constraint catching a record nested inside itself at any depth — `ex:hasElement`'s asymmetry and irreflexivity only cover one and two steps, and only under a reasoner.

SHACL runs in **two passes**, with RDFS inference on and off. Both are needed: the ontology declares `rdfs:domain` / `rdfs:range` on its properties, and under RDFS those axioms *manufacture* the typing that the type-asserting shapes exist to check. `ex:BuildsUponEndpointTypeShape` claims to catch "a node that uses buildsUpon but was never given any RecordHarm typing" — verified, it can only do so with inference off; with it on, the node is silently typed and reported by unrelated shapes instead. Versions through v3.0 ran the first pass only.

Plus one structural check in `scripts/validate.py` that neither OWL nor SHACL can express: every aspect in `RecordAspectScheme` must be targeted by at least one `PrimeHarm`. An aspect only composites attack means some composite is rooted in a prime that does not actually attack it.

Version History

v3.1 (Current)

- **Record-side structure:** `ex:hasElement` / `ex:elementOf` (mereology, aligned to `dcterms:hasPart`) and `ex:bearer` / `ex:bears` (where a record resides). `ex:Record` was previously a terminal node, leaving v3.0's own "elements include relations and adjacent metadata" commitment inexpressible.
- The **inside-agent / inside-community** distinction needs no new harm type and no new aspect — every case lands on an existing prime, so location is a parameter of the record. **Self-directed harm** (perpetrator = bearer: repression, self-deception) is newly expressible.
- `ex:Record`'s definition widened — "informational artifact" excluded memories and beliefs, though the harm definitions never did. `ex:Accessibility` documented as observer-relative, which v3.0's Suppression promotion already relied on.
- **SHACL now runs with inference on and off:** RDFS entailment from the ontology's own domain/range axioms was making the type-asserting shapes unable to fail. Added `ex:RecordShape` and `ex:RecordAcyclicShape`.

- Documented rather than fixed: detectability/reversibility are community-calibrated and wrong for inside-agent records; pure non-creation remains out of scope.

v3.0 — BREAKING

- **Breaking:** `ex:Suppression` changes class across an `owl:disjointWith` boundary, so a 2.3 consumer asserting `ex:Suppression a ex:CompositeHarm` gets an inconsistent graph. Migration table in [CHANGELOG.md](#).
- **Prime-set correction:** `ex:Suppression` promoted from `CompositeHarm` to a sixth `PrimeHarm`, and its `buildsUpon ex:Omission` edge removed as a category error — `Omission` excludes elements from a record, while a suppressed record is complete and unmodified. Found by a coverage diagnostic: no prime targeted `ex:Accessibility`.
- `ex:Omission` now states that a record's *elements* include its relations, ordering, and adjacent metadata, and targets `ex:Context`. Fragmentation and Decontextualization were both silently relying on this; it also absorbs "spurious association" without needing a new prime.
- `ex:Obfuscation` builds on `ex:Alteration` as well as `ex:Suppression`; definition now distinguishes a record's form from its surface content; reversibility corrected to `PartiallyReversible`.
- `ex:MiscontextualizationPattern` added; the false-context clause removed from `ex:Decontextualization`, which was carrying two harms on one `Omission` edge.
- Scope note on the deliberate absence of a confidentiality aspect; aspect-coverage regression guard added to `scripts/validate.py`.

v2.3

- **OWL 2 DL profile fix:** removed `owl:TransitiveProperty` from `ex:buildsUpon`. Transitivity is incompatible with the property also being asymmetric, irreflexive, and used in a cardinality restriction (all forbidden for "non-simple" properties in OWL 2 DL); v2.0–v2.2 were therefore OWL 2 Full despite the DL claim. Transitive closure remains available via the `ex:buildsUpon+` SPARQL path.

v2.2

- Added `HarmEvent` class for modeling specific occurrences
- Added `HarmPattern` class for empirical co-occurrences

- Introduced controlled vocabularies for detectability/reversibility
- Added inverse property `isBuiltUponBy`
- Separated SHACL shapes into companion file
- Added worked examples

v2.1

- Promoted Denial from CompositeHarm to 5th PrimeHarm
- Added SKOS typing to all harm instances
- Added `rdfs:isDefinedBy` to all harms

v2.0

- Replaced boolean flags with disjoint OWL classes
- Made `buildsUpon` transitive, asymmetric, irreflexive
- Promoted RecordAspect to proper SKOS vocabulary
- Eliminated duplicate aspect-subclasses

v1.0

- Initial release with 4 prime harms
- Basic taxonomy structure

Contributing

This ontology is part of the broader "thought framework" project exploring philosophical approaches to information integrity. Contributions welcome via:

1. Issue reports for conceptual inconsistencies
2. Pull requests for new harm types or patterns
3. Example data demonstrating real-world applications
4. Documentation improvements

License

The ontology and documentation are licensed **CC BY 4.0**

(<https://creativecommons.org/licenses/by/4.0/>) — permissive, reuse and adapt freely (including commercially), with attribution:

"Record Harm Ontology — epistemic-ontology.net" — <https://www.epistemic-ontology.net/record-harm>

The scripts (`scripts/`) are additionally available under the **MIT License**. See [LICENSE](#). Copyright © 2026 Ron Hinchley / epistemic-ontology.net.

Citation

If you use this ontology in academic work, please cite:

```
@misc{recordharmonontology2026,  
  title={Record Harm Ontology: A Formal Model of Ontological Attacks on Informatio  
  author={Hinchley, Ron},  
  year={2026},  
  version={3.1},  
  url={https://github.com/commuted/record-harm-ontology}  
}
```

Contact

- **Project:** Part of the thought framework collection
- **Repository:** <https://github.com/commuted/record-harm-ontology>
- **Issues:** <https://github.com/commuted/record-harm-ontology/issues>

Related Work

- **Unified Record Thesis** - Philosophical foundation for comprehensive record-keeping
- **AI Ethics Framework** - Single-party reconciliation through escalation analysis

- **Thought Escalation Plugin** - Conflict analysis and permanent front mapping

Technical Requirements

- **RDF Library:** rdflib (Python), Apache Jena (Java), or similar
- **SHACL Validator:** pyshacl, Apache Jena SHACL, or TopBraid
- **OWL Reasoner** (optional): HermiT, Pellet, or ELK for DL inference
- **SPARQL Endpoint** (optional): Fuseki, Virtuoso, or GraphDB

Acknowledgments

Conceptual modeling by Grok; ontology engineering and SHACL validation by Claude (Anthropic); philosophical framework development through extended dialogue in the thought framework project.